

Unpacking Malware - Qakbot

Qbot (also known as QakBot or QuackBot) is indeed a, persistent, and highly sophisticated banking trojan that has been consistently packed or packed in a new layer of obfuscation for many years to evade detection by security software

It is considered a "Swiss Army knife" for threat actors, operating not only as a banking trojan to steal credentials but also as a malware loader for ransomware

Sample: <https://malshare.com/sample.php?>

[action=detail&hash=112a64190b9a0f356880eebf05e195f4c16407032bf89fa843fd136da6f5d515](#)

File Identification

Field	Value
File Name	Qakbot
MD5	bce0df8721504d50f4497c0a0a2c090d
SHA-1	2c5b190d19f0f58e156bd1b28434701cea09cc23
SHA-256	112a64190b9a0f356880eebf05e195f4c16407032bf89fa843fd136da6f5d515
Vhash	026086551d5c1c1c5c551214z2500927z27z6030028fz
Authentihash	86da2bddaab0b1a3d6546dc35de29d8942856dc3eb8e64bd6be9e3dc097c2294
Imphash	829e83c1d2d988349a749b806aa9cfef
SSDEEP	12288 : q\lQq2wwLHqpVxTp5WK1QAPPV/Ykfgn6ggKh: u2wwT45xQwkfg93h
TLSH	T1F9D5CF27B560458BE7054A3684F6C9B026A1FEFD563520461EF0BE1BFBB1A934C11E8F
File Type	Win32 EXE (executable, windows, win32, pe, peexe)
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
TrID	• Win32 Executable MS Visual C++ (generic) (47.3%) • Win64 Executable (generic) (15.9%) • Win32 Dynamic Link Library (generic) (9.9%) • Win16 NE executable (generic) (7.6%) • Win32 Executable (generic) (6.8%)
DetectItEasy	• PE32 • Linker: Polink (2.50*) [GUI32,signed] • Sign tool: Windows Authenticode (2.0) [PKCS #7]
Magika	PEBIN
File Size	2.63 MB (2,755,088 bytes)

Sections

Index	Name	SHA-256 Hash	Entropy	File Ratio	Raw Size (Bytes)
0	.text	0917EF1437D686D73C90CE2823E761224AFF706BC2AFCF50FC14070A4AE5AB35	5.517	2.99 %	82,432
1	.rdata	CA63C02CAA8C0248F3F6A8291F23B0C2F64A6B933F9E7C99DF7E4B62762A7480	3.213	0.02 %	512
2	.data	126425A525E41E910B9358492365743A2330C7F260247AEA3F906E09B298490F	5.580	0.41 %	11,264
3	a2	AF3AB617E13B364CA364C8A8192346E14E55A2271662E40942D0BB415B1798B3	0.139	0.02 %	512
4	a3	0F2F59AFFEEC6B89C23DFF01782CBE898B57D866326E828C59521048AD4BEB5C	1.157	89.96 %	2,478,592
5	a32	60AE7E06BA7EA296D2D0A11FD046BA9098E1673CCB876B81C7E72752B4100862	4.845	0.04 %	1,024
6	a322	B76B11A36D44B4C17175B5AE7E018B975E79F7B02EEB6E44B4F143316343A33A	5.474	4.61 %	126,976
7	.rsrc	597AB4CED962CE1851893DF4A910CF2A54D7A0034166CBF1006CA9153966D6C	3.632	1.88 %	51,712

Key Observations

- **Space Efficiency:** The file has a total raw size of approximately **2.63 MB**, of which **2.36 MB** belongs solely to the **a3** section.
- **Entropy Alert:** Section **a3** shows extremely low entropy (**1.157**), indicating it is almost entirely redundant data (null bytes or repetitive padding), effectively "bloating" the file size.
- **Non-Standard PE Layout:** The sequential naming of **a2** through **a322** is a strong indicator of a custom packer or an obfuscation layer designed to make manual analysis more difficult.

Imports

Library (DLL)	Function Name	Binding	Location (VA)
KERNEL32.dll	Sleep	Implicit	0x000188F8
	GetLastError	Implicit	0x00018900
	GetModuleHandleW	Implicit	0x00018910
	LoadLibraryA	Implicit	0x00018924
	GetProcAddress	Implicit	0x00018934
	CreateMutexA	Implicit	0x00018946
	VirtualAlloc	Implicit	0x00018A8E
	WriteFile	Implicit	0x00018AAC
	WinExec	Implicit	0x000190EE
	CreateThread	Implicit	0x00018E26
USER32.dll	GetKeyState	Implicit	0x000193D4
	GetAsyncKeyState	Implicit	0x000194B2
	BlockInput	Implicit	0x00019450
	wsprintfW	Implicit	0x00019530
	SendMessageW	Implicit	0x0001953C
ADVAPI32.dll	RegOpenKeyExW	Implicit	0x000198E6
	RegSetValueExW	Implicit	0x000199E0
	CryptAcquireContextA	Implicit	0x0001994C
	CryptGenRandom	Implicit	0x0001999A
	CryptHashData	Implicit	0x00019A68
GDI32.dll	CreateSolidBrush	Implicit	0x00019664
	DeleteObject	Implicit	0x000196F6
	GetDeviceCaps	Implicit	0x0001985C
SHELL32.dll	ShellExecuteW	Implicit	0x00019828
	SHCreateProcessAsUserW	Implicit	0x00019AF8
SHLWAPI.dll	PathFindFileNameW	Implicit	0x00019B92
	PathCombineW	Implicit	0x00019BBC
ole32.dll	CoTaskMemFree	Implicit	0x00019B66

Library (DLL)	Binding	Import Count	Description
KERNEL32.dll	Implicit	146	Windows NT BASE API Client
USER32.dll	Implicit	40	Multi-User Windows USER API Client Library
GDI32.dll	Implicit	37	GDI Client Library
ADVAPI32.dll	Implicit	33	Advanced Windows 32 Base API
SHELL32.dll	Implicit	6	Windows Shell Library
SHLWAPI.dll	Implicit	3	Shell Light-weight Utility Library
ole32.dll	Implicit	2	Microsoft OLE for Windows

Analysis of Imports

Based on these specific function calls, the file possesses several notable capabilities:

- ## X32-DBG

The screenshot displays the Immunity Debugger interface with the following components:

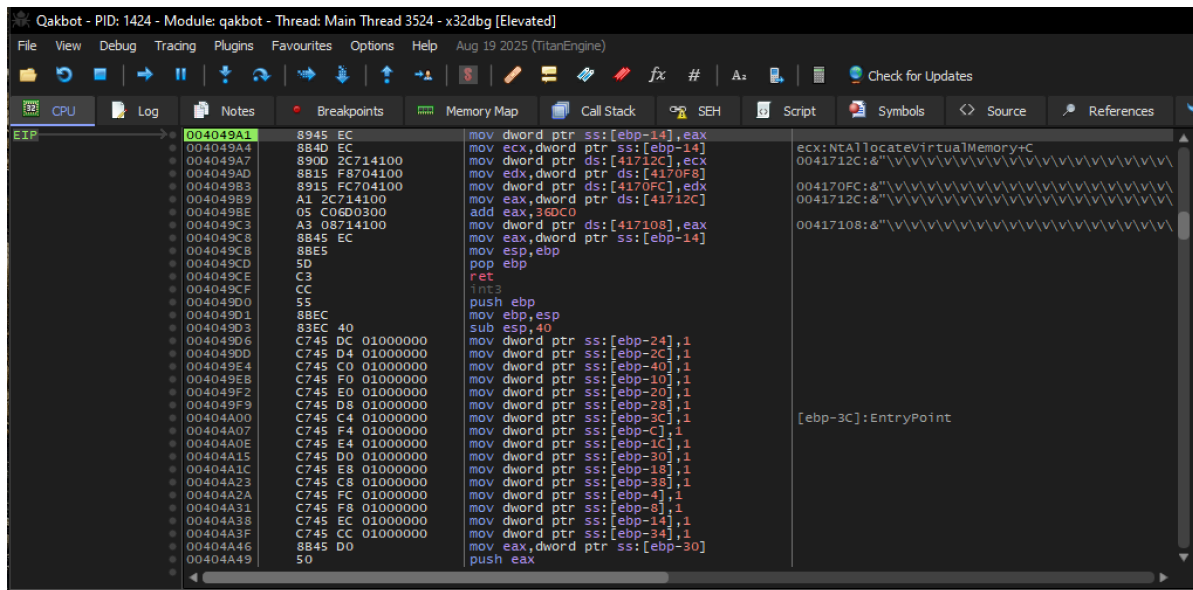
- Menu Bar:** File, View, Debug, Tracing, Plugins, Favourites, Options, Help.
- Toolbar:** Standard debugger controls (run, step, etc.) and a 'Check for Updates' button.
- Register Window:** Shows CPU registers (EAX, ECX, EDX, etc.) and their values.
- Memory Dump:** Displays a hex dump of memory starting at address 00401520. The data includes various byte sequences and some ASCII text.
- Assembly View:** Shows the disassembled instructions corresponding to the memory dump. Key instructions include:
 - `push ebp`
 - `mov ebp, esp`
 - `sub esp, 4`
 - `mov dword ptr ss:[ebp-4], 0`
 - `cmp dword ptr ss:[ebp-4], 0`
 - `je qakbot.40155E`
 - `push 1`
 - `call qakbot.404890`
 - `add esp, 4`
 - `push 1`
 - `call qakbot.404890`
 - `add esp, 4`
 - `push 1`
 - `call qakbot.404890`
 - `add esp, 4`
 - `push qakbot.404720C`
 - `call dword ptr ds:[LoadCursorFromFile]`
 - `mov dword ptr ss:[ebp-64], 64`
- Dialog Box:** A 'VirtualAlloc' dialog is open, prompting the user to 'Enter expression to follow...'. The 'Correct expression' button is highlighted, and the expression 'kernel32.VirtualAlloc' is entered.
- Register Window (Right):** Shows the values of registers EAX, ECX, EDX, EBP, ESP, ESI, EDI, EIP, EFLAGS, and LastStatus.
- Command Line:** Displays the command: `!test 0040155B qakbot:155B #958`.
- Bottom Panel:** Shows a list of memory dumps (Dump 1 to Dump 5) and a 'Watch' window.

Address	Disassembly	Comment
765DF9F0	8BFF	mov edi,edi
765DF9F2	55	push ebp
765DF9F3	8BEC	mov ebp,esp
765DF9F5	5D	pop ebp
765DF9F6	FF25 6C136476	jmp dword ptr ds:[<VirtualAlloc>]
765DF9FC	CC	int3
765DF9FD	CC	int3
765DF9FE	CC	int3
765DF9FF	CC	int3
765DFA00	CC	int3
765DFA01	CC	int3
765DFA02	CC	int3
765DFA03	CC	int3
765DFA04	CC	int3
765DFA05	CC	int3
765DFA06	CC	int3
765DFA07	CC	int3
765DFA08	CC	int3
765DFA09	CC	int3
765DFA0A	CC	int3
765DFA0B	CC	int3
765DFA0C	CC	int3
765DFA0D	CC	int3
765DFA0E	CC	int3
765DFA0F	CC	int3
765DFA10	8BFF	mov edi,edi
765DFA12	55	push ebp
765DFA13	8BEC	mov ebp,esp
765DFA15	8B55 0C	mov edx,dword ptr ss:[ebp+C]
765DFA18	33C0	xor eax,eax
765DFA1A	8B4D 08	mov ecx,dword ptr ss:[ebp+8]
765DFA1D	50	push eax
765DFA1E	6A 02	push 2
765DFA20	50	push eax

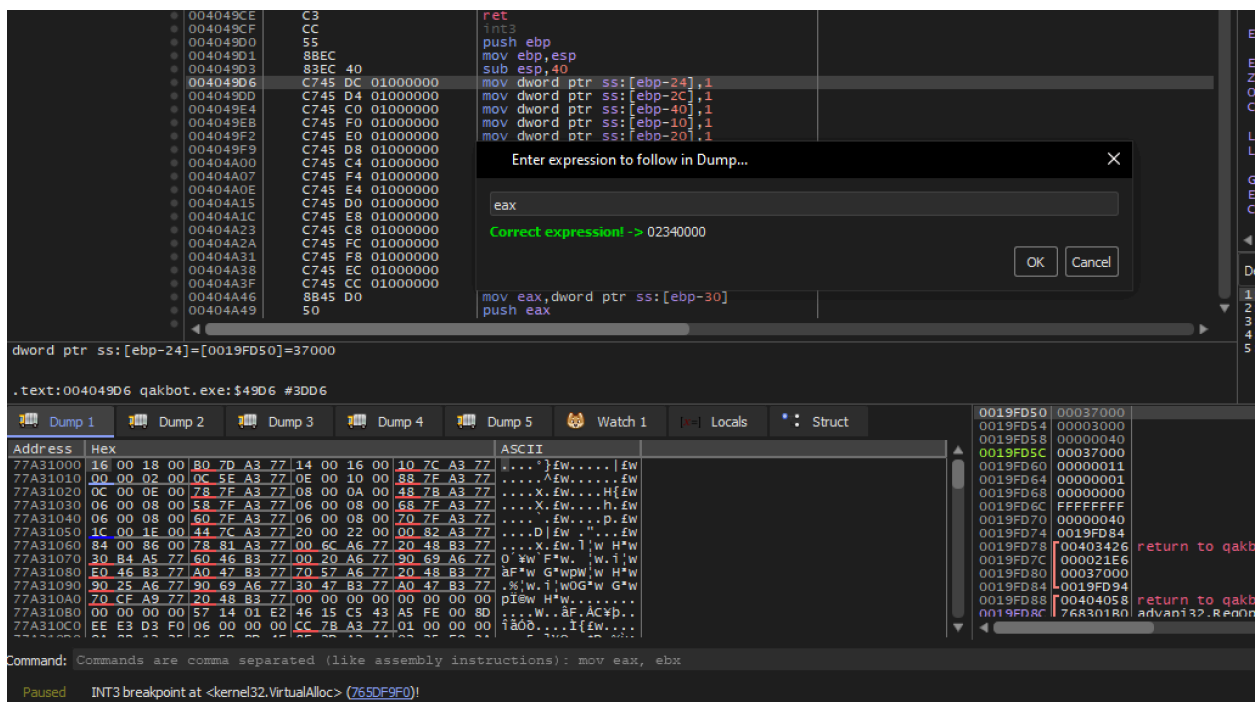
Set breakpoint, hit f9 until hit this breakpoint.

Address	Disassembly	Comment
765DF9F0	8BFF	mov edi,edi
765DF9F2	55	push ebp
765DF9F3	8BEC	mov ebp,esp
765DF9F5	5D	pop ebp
765DF9F6	FF25 6C136476	jmp dword ptr ds:[<VirtualAlloc>]
765DF9FC	CC	int3
765DF9FD	CC	int3
765DF9FE	CC	int3
765DF9FF	CC	int3
765DFA00	CC	int3
765DFA01	CC	int3
765DFA02	CC	int3
765DFA03	CC	int3
765DFA04	CC	int3
765DFA05	CC	int3
765DFA06	CC	int3
765DFA07	CC	int3
765DFA08	CC	int3
765DFA09	CC	int3
765DFA0A	CC	int3
765DFA0B	CC	int3
765DFA0C	CC	int3
765DFA0D	CC	int3
765DFA0E	CC	int3
765DFA0F	CC	int3
765DFA10	8BFF	mov edi,edi
765DFA12	55	push ebp
765DFA13	8BEC	mov ebp,esp
765DFA15	8B55 0C	mov edx,dword ptr ss:[ebp+C]
765DFA18	33C0	xor eax,eax
765DFA1A	8B4D 08	mov ecx,dword ptr ss:[ebp+8]
765DFA1D	50	push eax
765DFA1E	6A 02	push 2
765DFA20	50	push eax

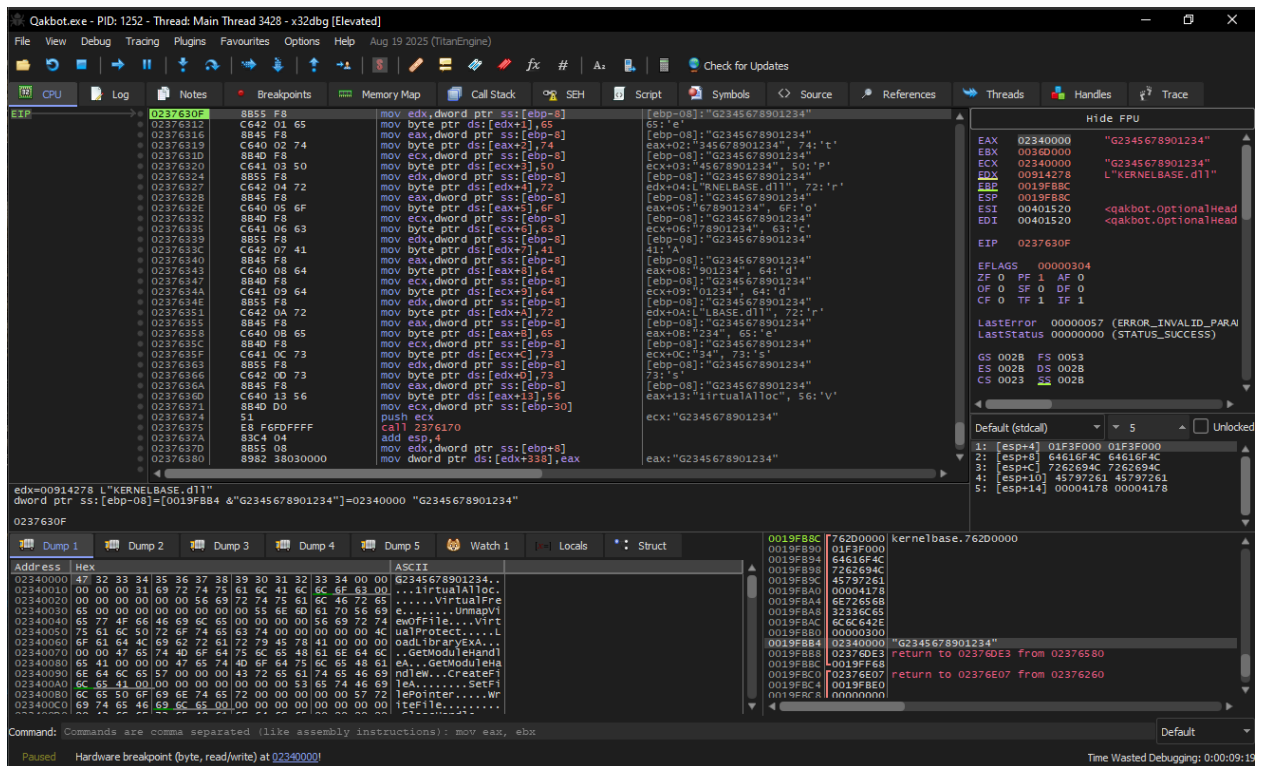
Then **ctrl + f9** then **f8** to get back to userspace.



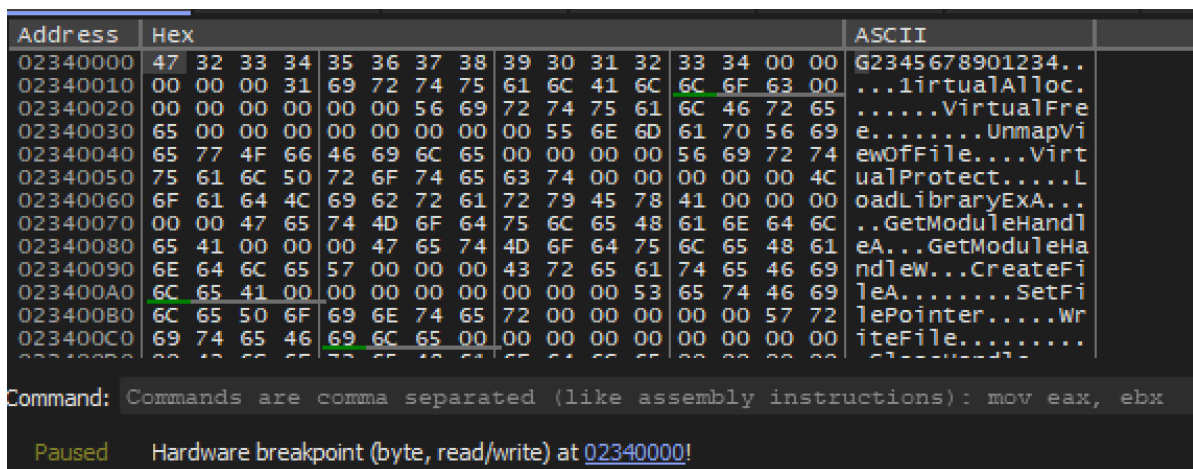
In hexdump **ctrl + g** follow eax.



Set breakpoint.



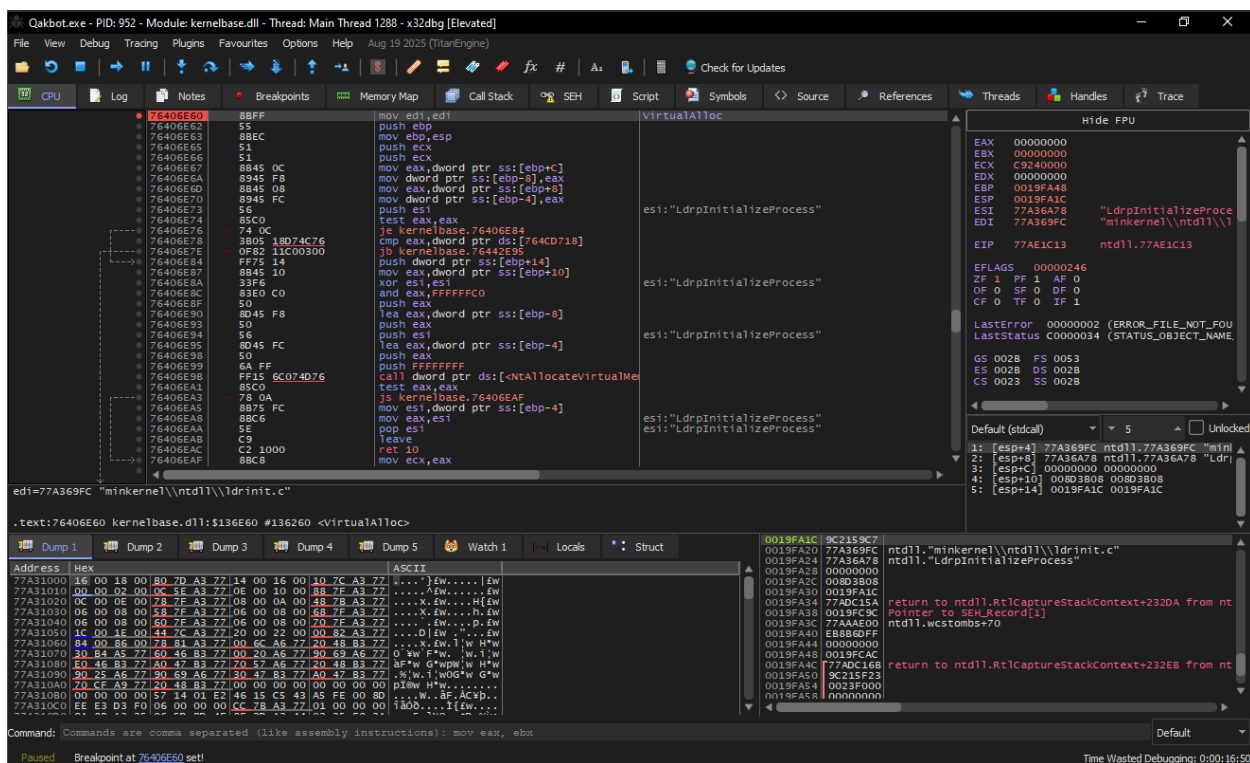
looks like an api lists.



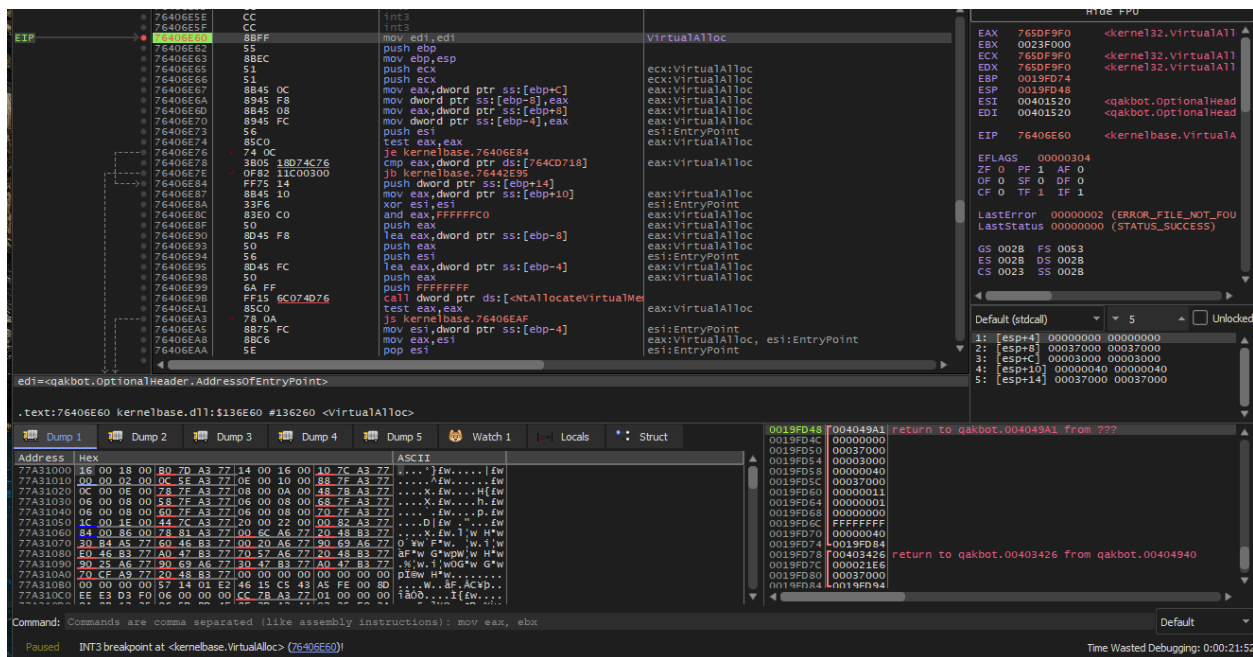
I restarted the program disable VirtualAlloc breakpoint.

765DF9F0	8BFF	mov edi,edi	VirtualAlloc
765DF9F2	55	push ebp	
765DF9F3	8BEC	mov ebp,esp	
765DF9F5	5D	pop ebp	
765DF9F6	FF25 6C136476	jmp dword ptr ds:[<VirtualAlloc>]	JMP.&VirtualAlloc
765DF9FC	CC	int3	
765DF9FD	CC	int3	
765DF9FE	CC	int3	
765DF9FF	CC	int3	
765DFA00	CC	int3	
765DFA01	CC	int3	
765DFA02	CC	int3	
765DFA03	CC	int3	
765DFA04	CC	int3	
765DFA05	CC	int3	
765DFA06	CC	int3	
765DFA07	CC	int3	
765DFA08	CC	int3	
765DFA09	CC	int3	
765DFA0A	CC	int3	
765DFA0B	CC	int3	
765DFA0C	CC	int3	
765DFA0D	CC	int3	
765DFA0E	CC	int3	
765DFA0F	CC	int3	
765DFA10	8BFF	mov edi,edi	
765DFA12	55	push ebp	
765DFA13	8BEC	mov ebp,esp	
765DFA15	8B55 0C	mov edx,dword ptr ss:[ebp+C]	
765DFA18	33C0	xor eax,eax	

Inside VirtualAlloc set Breakpoint, So Kernel32 Virtually calls Kernelbase Alloc

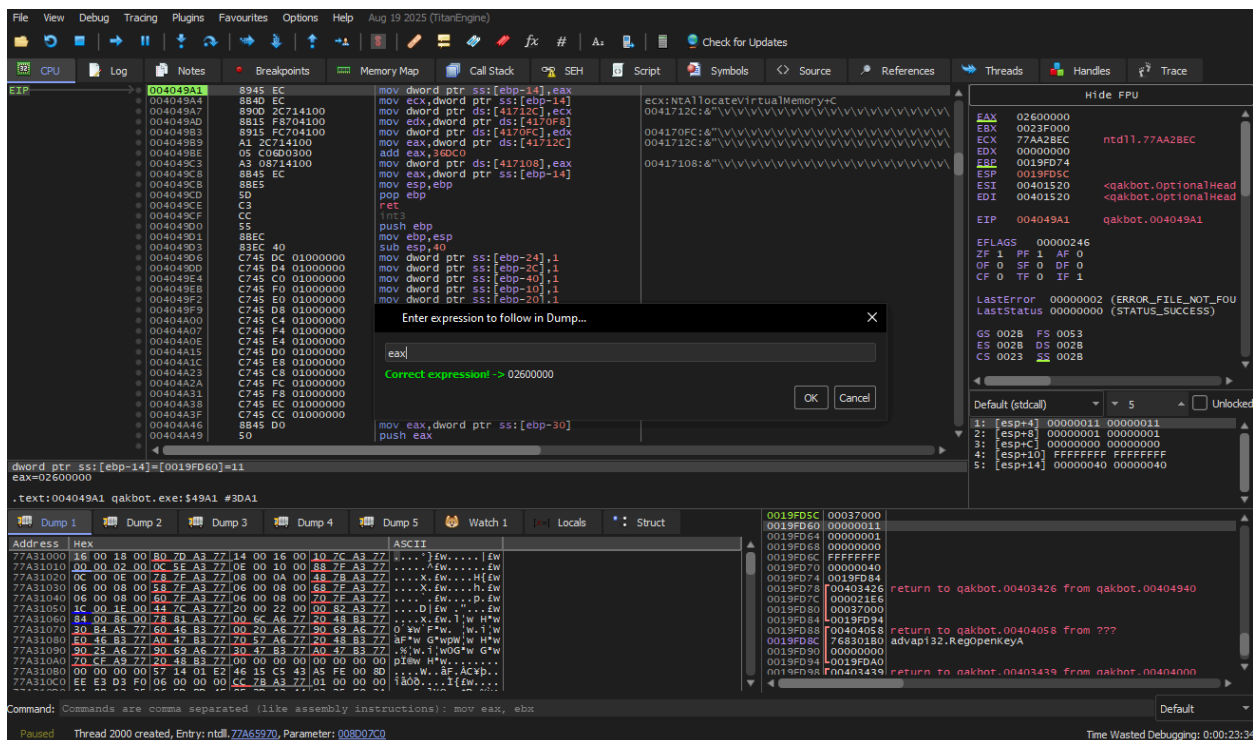


F9 hit the break point

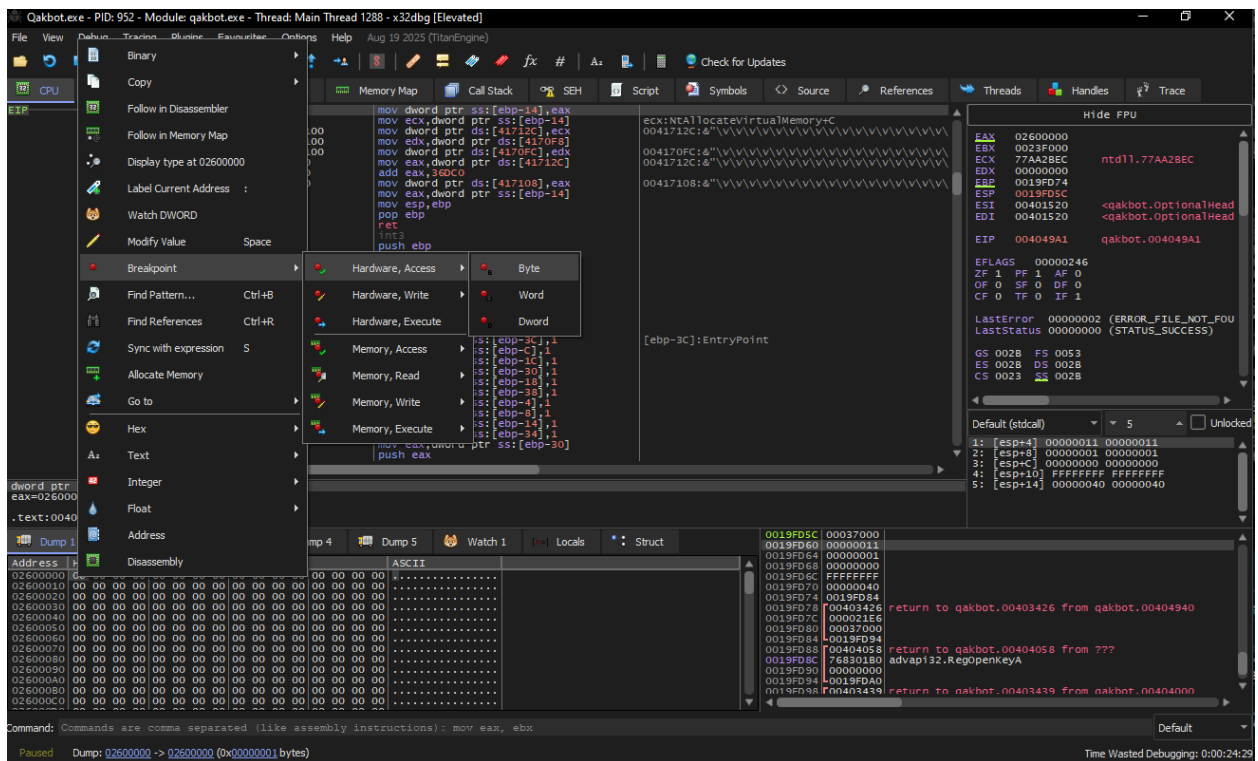


hit **ctrl + F9** and **F8**.

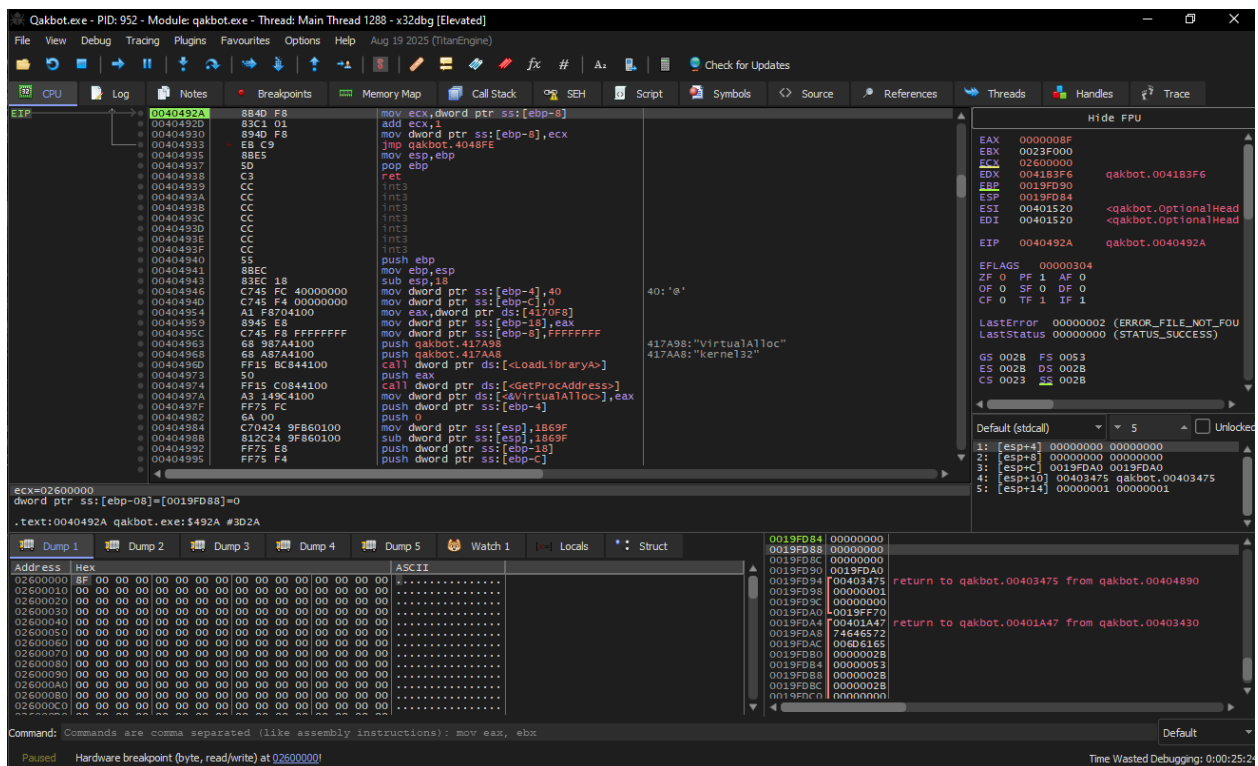
Then in memory dump **ctrl + g** Follow EAX.



Set break point



As before hit **F9** we see first byte **8F**



now hit **F9** until you see clear text in memory dump or a **VirtualAlloc**

We get the MZ header

The screenshot shows the Immunity Debugger interface for Qakbot.exe (PID: 4752). The CPU window displays assembly instructions at address 023A65FA, including `mov byte ptr ds:[ecx],al`. The register window shows `EAX 0000004D 'M'`. The memory dump window shows the MZ header starting at address 023B0000, with the first byte being 'M' (0x4D) and the second byte being 'Z' (0x5A). The command window shows the command `mov eax, ebx`.

scrolling a bit after mz header end of the header the first section starts from 400 hex.

The screenshot shows the Immunity Debugger interface for Qakbot.exe (PID: 4752). The CPU window displays assembly instructions at address 023B0000, including `mov byte ptr ds:[ecx],al`. The register window shows `EAX 0000004D 'M'`. The memory dump window shows the first section starting at address 023B0000, with the first byte being 'M' (0x4D) and the second byte being 'Z' (0x5A). The command window shows the command `mov eax, ebx`.

We can Verify it .text section begins with 400

property	value
section	section[0]
name	.text
section > sha256	0917EF1437D686D73C90CE2...
entropy	5.517
file > ratio (99.93%)	2.99 %
raw-address (begin)	0x00000400
raw-address (end)	0x00014600
raw-size (2753024 bytes)	0x00014200 (82432 bytes)
virtual-address (begin)	0x00001000
virtual-address (end)	0x0001508A
virtual-size (2751194 bytes)	0x0001408A (82058 bytes)

Dump Memory

The screenshot shows the x32dbg interface with Qakbot.exe loaded. The CPU register window displays the EAX register with the value 0x00000000. The memory dump window shows a list of memory addresses and their contents. A 'Save Memory Region' dialog box is open, showing the file name 'qakbot_02380000.bin' and the save type 'Binary files (*.bin)'. The dialog also shows a list of folders on the desktop, including 'PS_Transcripts', 'Tools', and 'fakenet_logs'.

Now Some calculations

Open new dump file in any pe analyzer tool.

#	Name	Relative address	Virtual size	File offset	Size	Flags	Info
0	.text	00000000	00001000	00000000	00000400		Header
1	.rdata	00001000	000099ca	00000400	00009a00	RE	(Compiler) Code Section
2	.data	0000b000	00004e68	00009e00	00005000	R	(Compiler) Read-only initialized Data Section (MS and Borland)
3	.rsrc	00010000	00002794	0000ee00	00000600	RW	(Compiler) Data Section
4	.reloc	00013000	00025fdc	0000f400	00026000	R	(Compiler) Resource section
5							(Compiler) Relocations Section

Let's focus on .reloc end section of the binary.

Fileoffset + Size 35400 + C00 = 36000

Hex Calculator

Hex calculator

Decimal to hex

Hex to decimal

35400

16

add (+)

C00

16

= Calculate

× Reset

Hex result

36000

16

Decimal result

221184

10

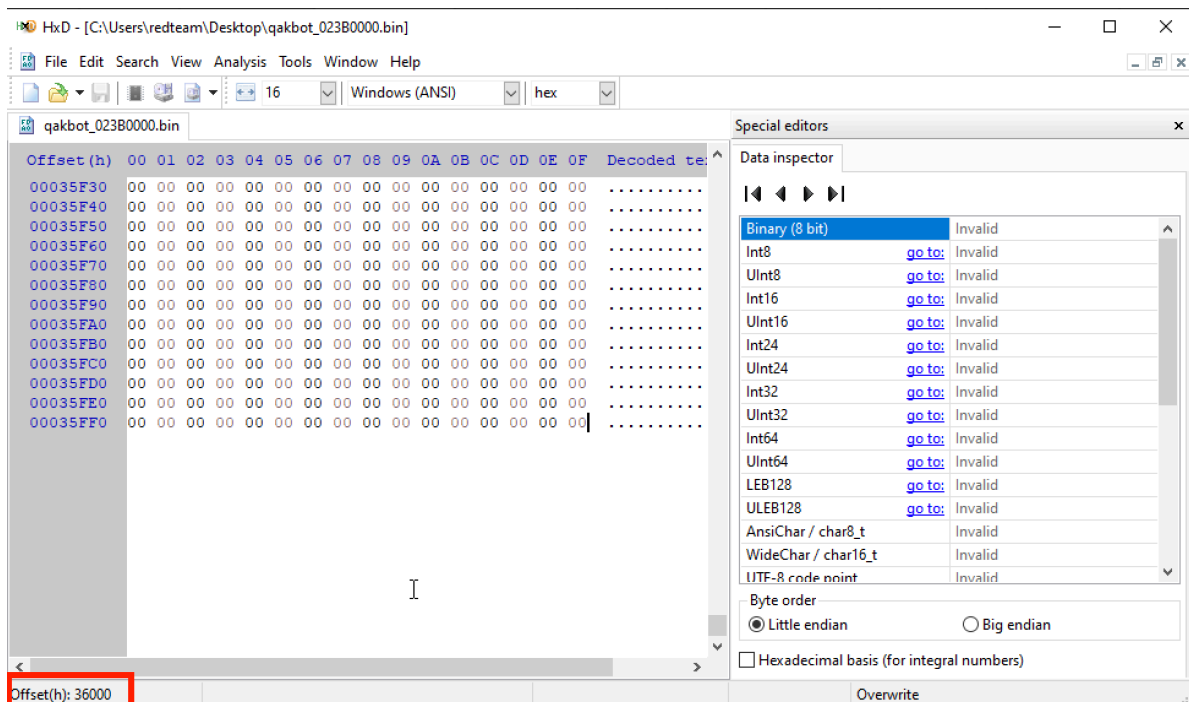
Binary result

11011000000000000000

2

☐ Digit grouping

We can verify offset 36000 it's point to the end of the file.



So we don't need any cleanup.

qakbot_023B0000.bin

Basic properties

Property	Value
MD5	58e1c32eeb0130da19625e55ee48cf1e
SHA-1	00ae1c5066f67e5e71285de99bea8d8b7085743
SHA-256	f5ff6dbf5206cc2db098b41f5af14303f6dc43e36c5ec02604a50d5cfecf4790
Imphash	f83b544e96ab46c08e00b6dc80fbb352
Vhash	025056657d15755198z4506033z67z33z4fz
Rich Header Hash	39b9a974374727230098b3403672606
SSDEEP	3072:4FCXMfyhFPZ8H7kjiIcKozOMeNjwUJCFufWcxQvAKChQztvWZZ0tyFb8e:LXFPF6HWriMeN2rJCyWVDhM55
TLSH	T11324F1017293E5F8DA4600390A49117726B4EE312C738647F2D077BD69F689FDA35AC2
File Type	Win32 EXE (PE32 executable, GUI)
Compiler	Microsoft Visual C/C++ (16.00.40219) [Visual Studio 2010]
Linker	Microsoft Linker (10.00) with LTCG (Link Time Code Generation)
File Size	216.00 KB (221,184 bytes)

PE section

Index	Name	Entropy	File Ratio	Raw Size	Virtual Size	Raw Address (Begin)	Virtual Address
0	.text	6.386	17.82 %	39,424	39,370	0x00000400	0x00001000
1	.rdata	7.515	9.26 %	20,480	20,072	0x00009E00	0x0000B000
2	.data	4.390	0.69 %	1,536	10,132	0x0000EE00	0x00010000
3	.rsrc	7.863	70.37 %	155,648	155,612	0x0000F400	0x00013000
4	.reloc	5.475	1.39 %	3,072	2,678	0x00035400	0x00039000

Key Observations for the Decrypted File

- **Resource Dominance:** The `.rsrc` section now accounts for over **70%** of the file. Its extremely high entropy (**7.863**) suggests it contains compressed data, encrypted payloads, or high-resolution media.
- **Normalized Naming:** Unlike the encrypted version (which used non-standard names like `a2`, `a3`), this file uses standard Microsoft compiler section names (`.text`, `.rdata`, etc.), confirming a successful "unpacking" or "decryption."
- **Data Expansion:** In section `[2]` (`.data`), the virtual size (**10,132 bytes**) is significantly larger than the raw size (**1,536 bytes**). This is typical for uninitialized data or variables that expand in memory once the program is loaded.
- **Code Complexity:** The `.text` section (where the executable code resides) has an entropy of **6.386**, which is standard for a compiled C++ application.

Decrypted Import Address Table Imports

Library (DLL)	Functions
KERNEL32.dll	<code>GetLastError</code> , <code>GetProcAddress</code> , <code>LoadLibraryA</code> , <code>lstrcpw</code> , <code>GetModuleHandleA</code> , <code>CloseHandle</code> , <code>GetCurrentProcessId</code> , <code>GetEnvironmentVariableW</code> , <code>lstrlenA</code> , <code>WideCharToMultiByte</code> , <code>lstrcatA</code> , <code>GetEnvironmentVariableA</code> , <code>MultiByteToWideChar</code> , <code>lstrlenW</code> , <code>lstrcatW</code> , <code>lstrcpyA</code> , <code>HeapAlloc</code> , <code>HeapFree</code> , <code>HeapCreate</code> , <code>VirtualAlloc</code> , <code>GetFileSize</code> , <code>lstrcmpiA</code> , <code>GetModuleFileNameA</code> , <code>GetThreadContext</code> , <code>GetCurrentProcess</code> , <code>CreateEventA</code> , <code>LoadLibraryW</code> , <code>TerminateProcess</code> , <code>DeleteFileW</code> , <code>ResumeThread</code> , <code>ExpandEnvironmentStringsW</code> , <code>GetComputerNameW</code> , <code>GetVolumeInformationW</code> , <code>ReleaseMutex</code> , <code>GetExitCodeProcess</code> , <code>GetSystemTimeAsFileTime</code> , <code>SetEnvironmentVariableW</code> , <code>GetTickCount</code> , <code>GetModuleFileNameW</code> , <code>GetSystemInfo</code> , <code>SetEnvironmentVariableA</code> , <code>GetVersionExA</code> , <code>GetWindowsDirectoryW</code> , <code>SetEvent</code> , <code>OpenEventA</code> , <code>CopyFileW</code> , <code>TerminateThread</code> , <code>CreateThread</code> , <code>GetFileAttributesA</code> , <code>GetFileAttributesW</code> , <code>GetCurrentThread</code> , <code>LocalAlloc</code> , <code>GetLocalTime</code> , <code>LocalFree</code> , <code>lstrcpyW</code> , <code>CreateDirectoryW</code> , <code>SleepEx</code> , <code>WaitForSingleObject</code> , <code>FreeLibrary</code> , <code>GetDriveTypeW</code> , <code>lstrcmpA</code> , <code>GetCommandLineW</code> , <code>ExitProcess</code> , <code>lstrcpyW</code> , <code>Sleep</code> , <code>SystemTimeToFileTime</code> , <code>GetSystemTime</code> , <code>GetModuleHandleW</code> , <code>CreateMutexA</code>
ADVAPI32.dll	<code>RegOpenKeyExW</code> , <code>RegEnumValueW</code> , <code>RegDeleteValueW</code> , <code>RegQueryInfoKeyW</code> , <code>LookupAccountNameW</code> , <code>EqualSid</code> , <code>SetServiceStatus</code> , <code>RegUnLoadKeyW</code> , <code>RegLoadKeyW</code> , <code>ConvertSidToStringSidW</code> , <code>RegSetValueExW</code> , <code>RegQueryValueExW</code> , <code>SetSecurityDescriptorDacl</code> , <code>InitializeSecurityDescriptor</code> , <code>GetTokenInformation</code> , <code>RegisterServiceCtrlHandlerA</code> , <code>StartServiceCtrlDispatcherA</code> , <code>RegCloseKey</code> , <code>SetFileSecurityW</code> , <code>OpenProcessToken</code> , <code>GetSidSubAuthority</code> , <code>OpenThreadToken</code> , <code>GetSidSubAuthorityCount</code> , <code>LookupAccountSidW</code> , <code>CreateProcessAsUserW</code>
USER32.dll	<code>CharUpperBuffA</code> , <code>MessageBoxA</code> , <code>GetClassNameA</code> , <code>CharUpperBuffW</code>
SHELL32.dll	<code>SHGetFolderPathW</code> , <code>CommandLineToArgvW</code> , <code>ShellExecuteW</code>
ole32.dll	<code>CoInitialize</code> , <code>CoInitializeEx</code> , <code>CoInitializeSecurity</code> , <code>CoSetProxyBlanket</code> , <code>CoUninitialize</code> , <code>CoCreateInstance</code>
NETAPI32.dll	<code>NetApiBufferFree</code> , <code>NetUserEnum</code> , <code>NetGetDCName</code>
SETUPAPI.dll	<code>SetupDiEnumDeviceInfo</code> , <code>SetupDiDestroyDeviceInfoList</code> , <code>SetupDiGetClassDevsA</code> , <code>SetupDiGetDeviceRegistryPropertyA</code>
msvcrt.dll	<code>_vsnprintf</code> , <code>_ltoa</code> , <code>_except_handler3</code> , <code>memset</code> , <code>_vsnprintf</code> , <code>memcpy</code>
USERENV.dll	<code>GetUserProfileDirectoryW</code>

Critical Behavioral Indicators

Analyzing these specific imports reveals the binary's core capabilities:

- **Process Hollowing/Injection:** The combination of `GetThreadContext`, `VirtualAlloc`, and `ResumeThread` is a classic signature for process hollowing, where the binary injects code into a suspended legitimate process.

- **Privilege Escalation & Identity:** Imports like `OpenProcessToken`, `GetTokenInformation`, and `CreateProcessAsUserW` suggest the ability to manipulate user tokens or run tasks with administrative/SYSTEM privileges.
- **Network Enumeration:** `NetUserEnum` and `NetGetDCName` indicate that the file is designed to query domain controllers and list users on a network, common in reconnaissance phases.
- **Persistence:** `StartServiceCtrlDispatcherA` and `SetServiceStatus` show the binary can operate as a Windows Service, allowing it to start automatically with the OS.
- **Hardware Profiling:** The `SETUPAPI.dll` imports suggest it is looking for specific hardware or drivers, possibly for environment keying or anti-VM checks.

Sample 2

Sample: [https://malshare.com/sample.php?](https://malshare.com/sample.php?action=detail&hash=1042f400ed776bc5d2c68becb386fb2ef3116417f96a67c14e8ca5b421ae7bc9)

[action=detail&hash=1042f400ed776bc5d2c68becb386fb2ef3116417f96a67c14e8ca5b421ae7bc9](https://malshare.com/sample.php?action=detail&hash=1042f400ed776bc5d2c68becb386fb2ef3116417f96a67c14e8ca5b421ae7bc9)

Property	Value
MD5	ce6d60aefd0bba037a50186818cc23cb
SHA-1	423155bd783bd5209f4ff05c23159fd75e2f4e5d
SHA-256	1042f400ed776bc5d2c68becb386fb2ef3116417f96a67c14e8ca5b421ae7bc9
Vhash	06508675555551d5f5510c1802007900ff6z156z1e0300fffz
Authentihash	1eaac19b0627e9e53c1df7f9671718f5fc161355d78104950abc96bdf8915030
Imphash	09516c081315a6de933d15f22a94d805
SSDEEP	12288:WTZ3ESbJAzTU+HjLI2UxI3JacfhLwVSZR0mjieAqTVc2xrW/f:AfJ8ThHjLZ3JxhL57ueAUvcarW/f
TLSH	T140E4CEB26BE40C26D69ACA7541611E177067FF0D2FC0580B6DC8E63BDA12BD98CD1E48
File Type	Win32 EXE (PE32 executable, GUI)
TrID	Win32 Executable MS Visual C++ (47.3%), Win64 Executable (15.9%)
DetectItEasy	PE32; Sign tool: Windows Authenticode (2.0)
File Size	648.52 KB (664,080 bytes)

PE section

Index	Name	SHA-256 (Full)	Entropy	File Ratio	Raw Size (Bytes)
0	.text	4EB5A21D4AA7DE4C2FCF4124FF224378F597DAF0B205E2F2680757BC46D74AA3	7.238	80.11%	531,968
1	.rdata4	1E8428ACD7003A0DF93471CE69B1CFD9A4A50D6D148AB95AB5E0E16C0FE14FB7	5.903	0.08%	512
2	.rdata3	1E8428ACD7003A0DF93471CE69B1CFD9A4A50D6D148AB95AB5E0E16C0FE14FB7	5.903	0.08%	512
3	.rdata2	1E8428ACD7003A0DF93471CE69B1CFD9A4A50D6D148AB95AB5E0E16C0FE14FB7	5.903	0.08%	512
4	.rdata	B17141BD9DEB3C6FBA87E5CE806D4CC11A98F8A5A4CB699296D4EBA8E608ADA2	3.378	0.08%	512
5	.data	57EA9046FE23666E1C9AA5C428B274D4F340B0D8FE6411D788E0ED570744E5D7	5.513	18.58%	123,392
6	.rdata5	1E8428ACD7003A0DF93471CE69B1CFD9A4A50D6D148AB95AB5E0E16C0FE14FB7	5.903	0.08%	512
7	.rsrc	716AEBF3D58674C078A9F93F3A5D020AB0E7D33A072DBCDEC0E43A9C4C2F7ED7	5.903	0.62%	4,096

Memory and Execution offsets

Item	Value	Section Association
Entry Point	0x00082630	Falls outside standard .text mapping
Base of Code	0x00001000	.text
Base of Data	0x00083000	.rdata4
Import Directory	0x0009F870	.data
Resource Directory	0x000A7000	.rsrc
Import Address Table	0x000A06D4	.data

- **Identical Sections:** Sections `.rdata4`, `.rdata3`, `.rdata2`, and `.rdata5` share the **exact same SHA-256 hash**. This indicates these sections are likely duplicates or placeholders filled with the same static data.
- **High Code Entropy:** The `.text` section has an entropy of **7.238**. This is quite high for standard executable code and often suggests the code is either compressed or contains an embedded encrypted payload.
- **Self-Modifying / W+X Flag:** Section `.rdata5` is flagged for **Execute and Write** permissions and is marked as **self-modifying**. This is a significant indicator of a packer or a polymorphic engine.
- **Entry Point Anomaly:** The Entry Point is at `0x00082630`, which places it far beyond the `.text` section's virtual end. This suggests the execution starts in a later section (likely the self-modifying `.rdata5` or a custom tail), a common trait of packed malware.

Imports

Library (DLL)	Notable Functions
KERNEL32.dll	<code>VirtualAlloc</code> , <code>VirtualProtect</code> , <code>WriteProcessMemory</code> , <code>ReadProcessMemory</code> , <code>CreateProcessW</code> , <code>WinExec</code> , <code>CreateThread</code> , <code>Module32FirstW</code> , <code>GetThreadContext</code> , <code>SetThreadPriority</code> , <code>SleepEx</code> , <code>GetTickCount</code> , <code>QueryPerformanceFrequency</code> , <code>MapViewOfFile</code> , <code>CreateFileMappingW</code>
USER32.dll	<code>SetWindowsHookExW</code> , <code>GetAsyncKeyState</code> , <code>GetKeyState</code> , <code>GetForegroundWindow</code> , <code>OpenDesktopW</code> , <code>AttachThreadInput</code> , <code>keybd_event</code> , <code>OpenClipboard</code> , <code>GetClipboardData</code> , <code>EmptyClipboard</code> , <code>EnumDisplayMonitors</code>
ADVAPI32.dll	<code>RegCreateKeyExA</code> , <code>RegSetValueExA</code> , <code>RegDeleteValueA</code> , <code>RegDeleteKeyA</code> , <code>RegOpenKeyExA</code> , <code>RegFlushKey</code> , <code>RegQueryInfoKeyA</code>
GDI32.dll	<code>BitBlt</code> , <code>StretchBlt</code> , <code>CreateCompatibleBitmap</code> , <code>SelectObject</code> , <code>CreateCompatibleDC</code> , <code>GetDeviceCaps</code> , <code>DeleteObject</code>
SHELL32.dll	<code>ShellExecuteExW</code> , <code>SHCreateProcessAsUserW</code> , <code>SHChangeNotify</code> , <code>SHGetPathFromIDListW</code> , <code>DoEnvironmentSubstA</code>
COMCTL32.dll	<code>ImageList_BeginDrag</code> , <code>ImageList_DragMove</code> , <code>ImageList_DragEnter</code> , <code>ImageList_EndDrag</code> , <code>DPA_DeleteAllPtrs</code>
ole32.dll	<code>CoCreateInstance</code> , <code>CoInitializeEx</code> , <code>CoUninitialize</code> , <code>DoDragDrop</code>

MITRE ATT&CK Mapping

Tactic	Technique	ID	Indicators (API/Logic)
Persistence	Boot or Logon Autostart Execution: Registry Run Keys	T1547.001	<code>RegCreateKeyExA</code> , <code>RegSetValueExA</code>
Privilege Escalation	Process Injection: Process Hollowing	T1055.012	<code>VirtualAllocEx</code> , <code>WriteProcessMemory</code> , <code>ResumeThread</code> , <code>GetThreadContext</code>
Defense Evasion	Obfuscated Files or Information: Software Packing	T1027.002	High entropy in <code>.text</code> (7.238), Self-modifying <code>.rdata5</code> section
	Virtualization/Sandbox Evasion: Time-Based Evasion	T1497.003	<code>QueryPerformanceFrequency</code> , <code>GetTickCount</code>
	Indicator Removal: File Deletion	T1070.004	<code>DeleteFileW</code> , <code>RemoveDirectoryW</code>
Discovery	System Information Discovery	T1082	<code>GetVersionExW</code> , <code>GetSystemInfo</code> , <code>GlobalMemoryStatusEx</code>
	Process Discovery	T1057	<code>Module32FirstW</code>
Collection	Input Capture: Keylogging	T1056.001	<code>GetAsyncKeyState</code> , <code>GetKeyState</code>
	Screen Capture	T1113	<code>BitBlt</code> , <code>StretchBlt</code> , <code>CreateCompatibleDC</code>
	Archive Collected Data: Clipboard Data	T1115	<code>OpenClipboard</code> , <code>GetClipboardData</code>
Execution	Shared Modules	T1129	Extensive use of <code>LoadLibraryW</code> and <code>GetProcAddress</code>

Initial Observations

- **Signature:** Unlike the previous samples, this one explicitly mentions **Windows Authenticode**, indicating the file is (or claims to be) digitally signed.
- **Size:** At **~648 KB**, it is significantly smaller than your first sample's encrypted state but larger than the decrypted payload.
- **Tooling:** TrID suggests a high likelihood of **MS Visual C++** origin.

Technical Summary:

This binary is a **digitally signed** Win32 executable that exhibits strong characteristics of a **packed surveillance tool** or **Remote Access Trojan (RAT)**.

1. Delivery & Structure

- **Packing Signature:** The high entropy in `.text` (**7.238**) and the existence of a **self-modifying, executable** section (`.rdata5`) indicate the core logic is encrypted or compressed.
- **Execution Anomaly:** The program starts at an unusual Entry Point (`0x00082630`) located outside the main code section, which is a classic indicator of a packer stub.
- **Size:** At **648 KB**, it is compact enough for quick delivery while containing significant functionality.

2. Core Capabilities (via Imports)

- **Code Injection:** It can manipulate other processes using `WriteProcessMemory` and `VirtualProtect`, likely to hide its presence within legitimate system tasks.
- **Information Theft:**
 - **Keylogging:** Monitors background keystrokes (`GetAsyncKeyState`).
 - **Clipboard Stealing:** Accesses and clears user clipboard data (`OpenClipboard`).
 - **Screen Grabbing:** Capable of capturing the desktop via GDI drawing functions.
- **Persistence:** Extensive Registry access (`ADVAPI32.dll`) suggests it modifies system "Run" keys to ensure it starts automatically upon reboot.

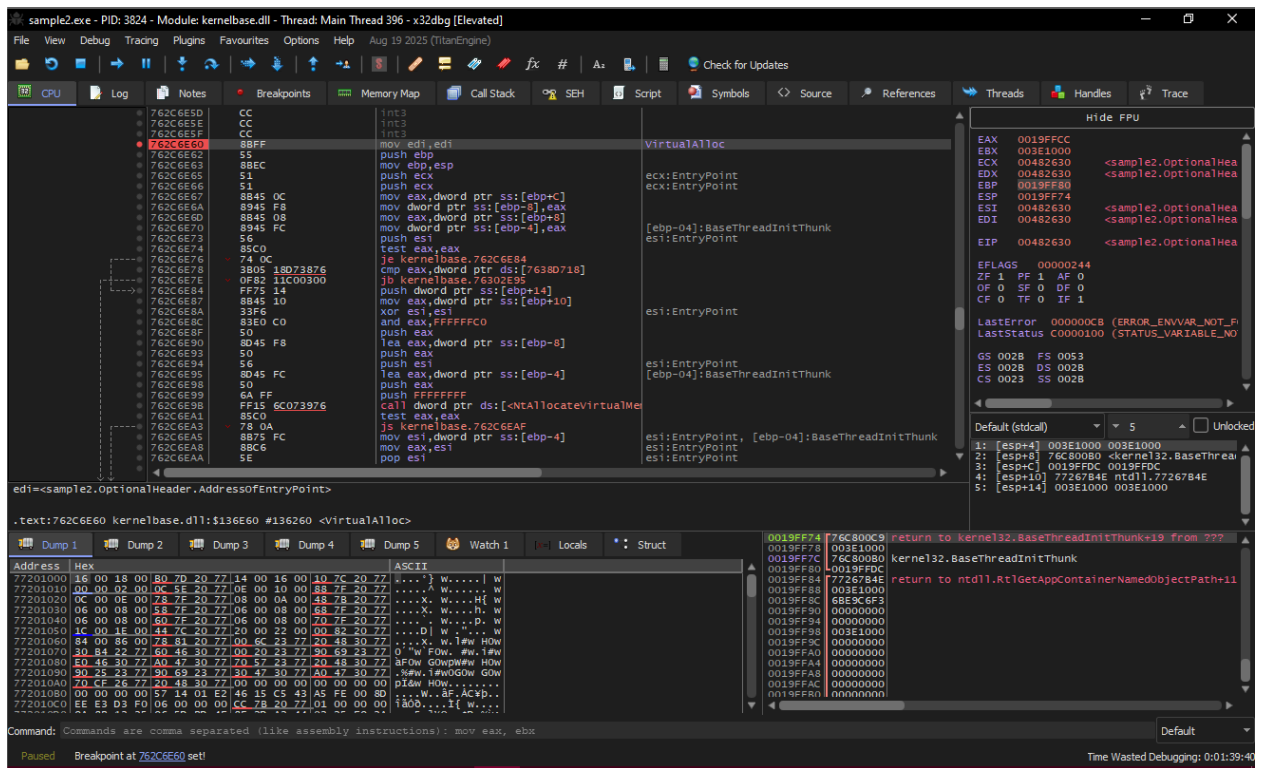
3. Operational Risk

The combination of **Keylogging**, **Clipboard access**, and **Screen capture** suggests this sample is designed for **credential harvesting** and user monitoring. Its ability to inject code further increases the risk of it evading standard task manager detection.

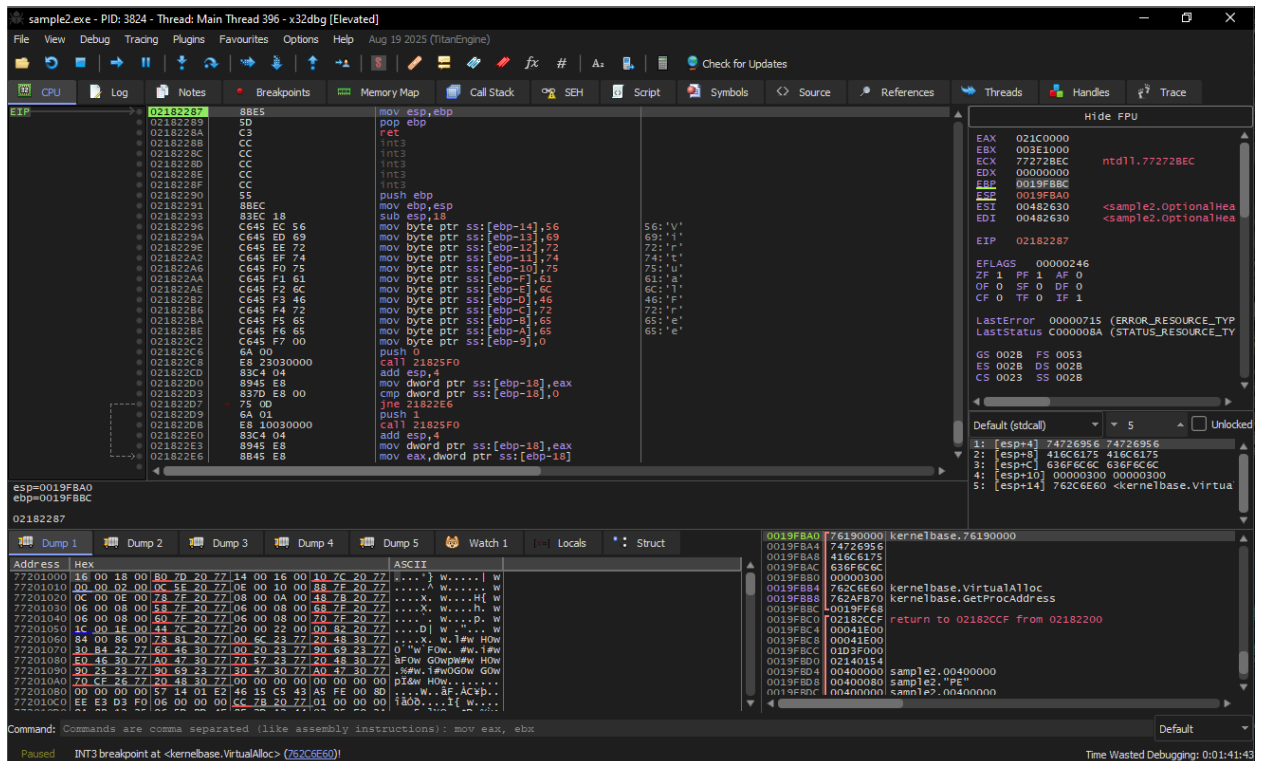
Unpacking

X32 DBG

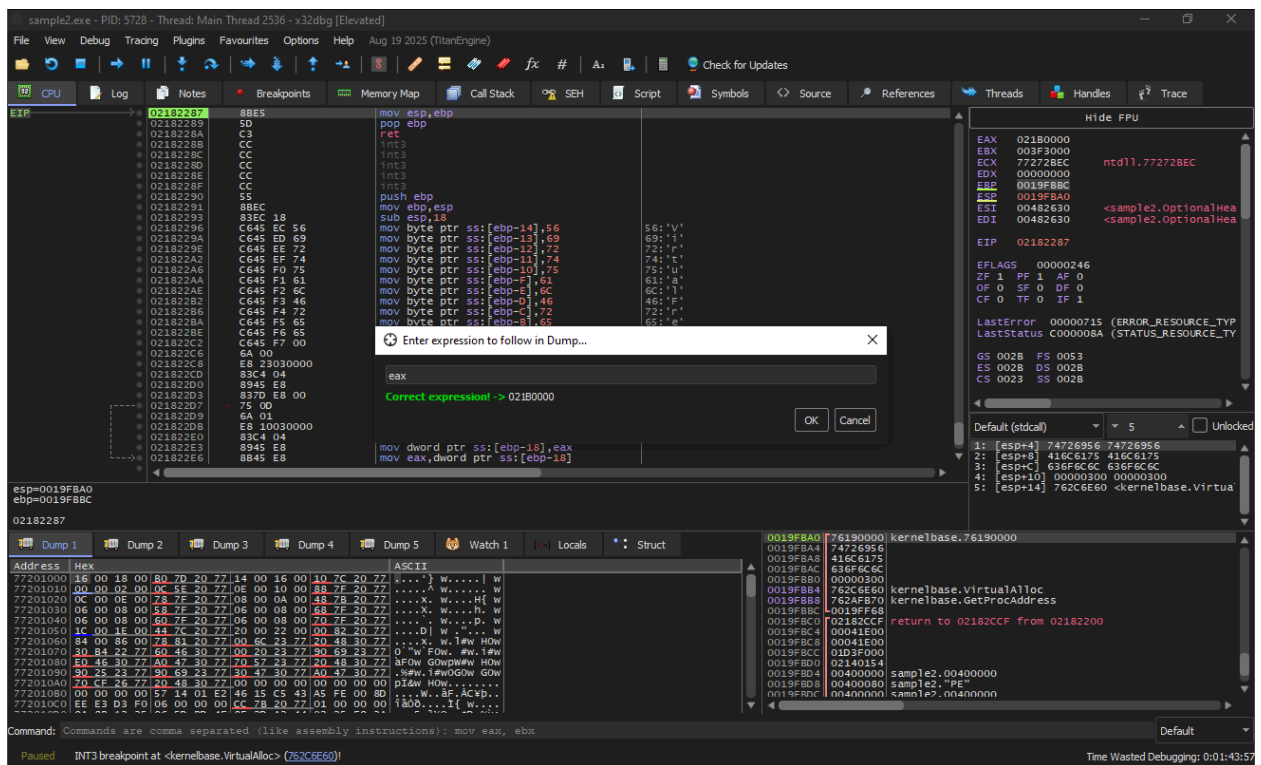
Setting breakpoint on VirtualAlloc



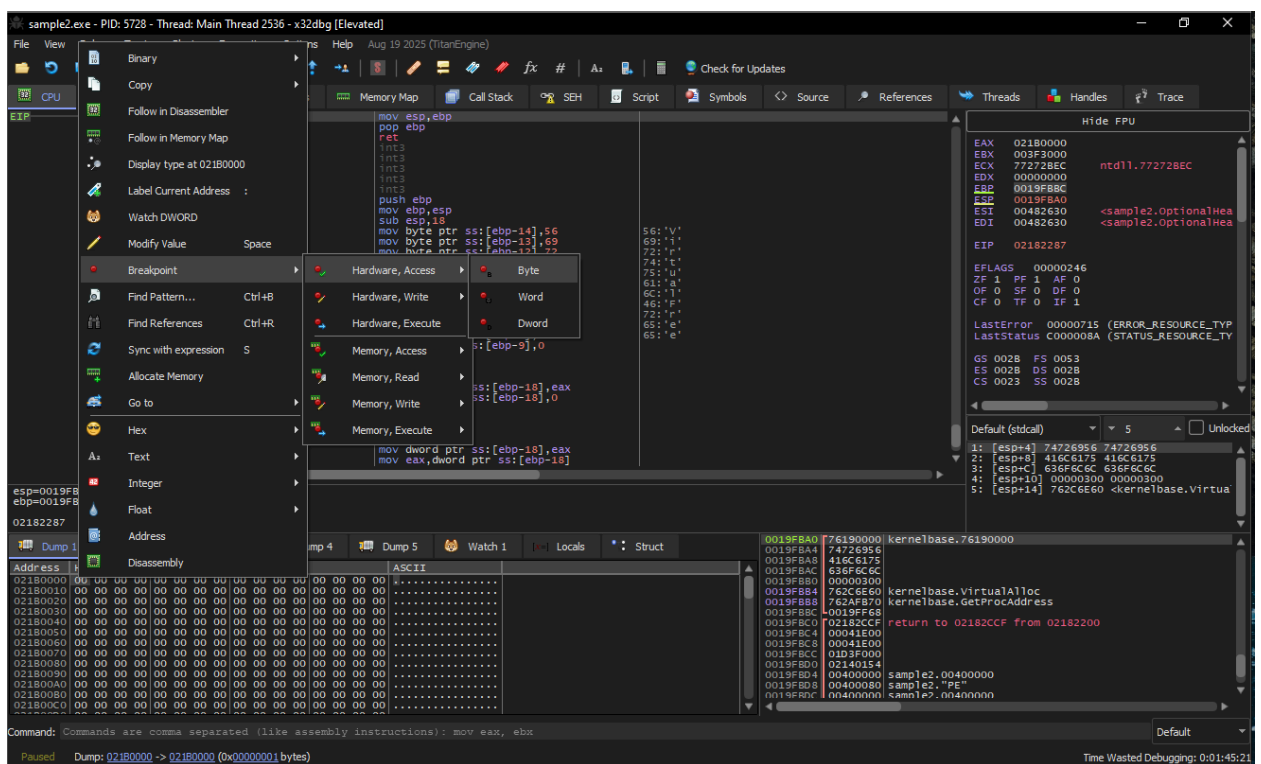
After hitting the breakpoint **ctrl + f9** and **f8**



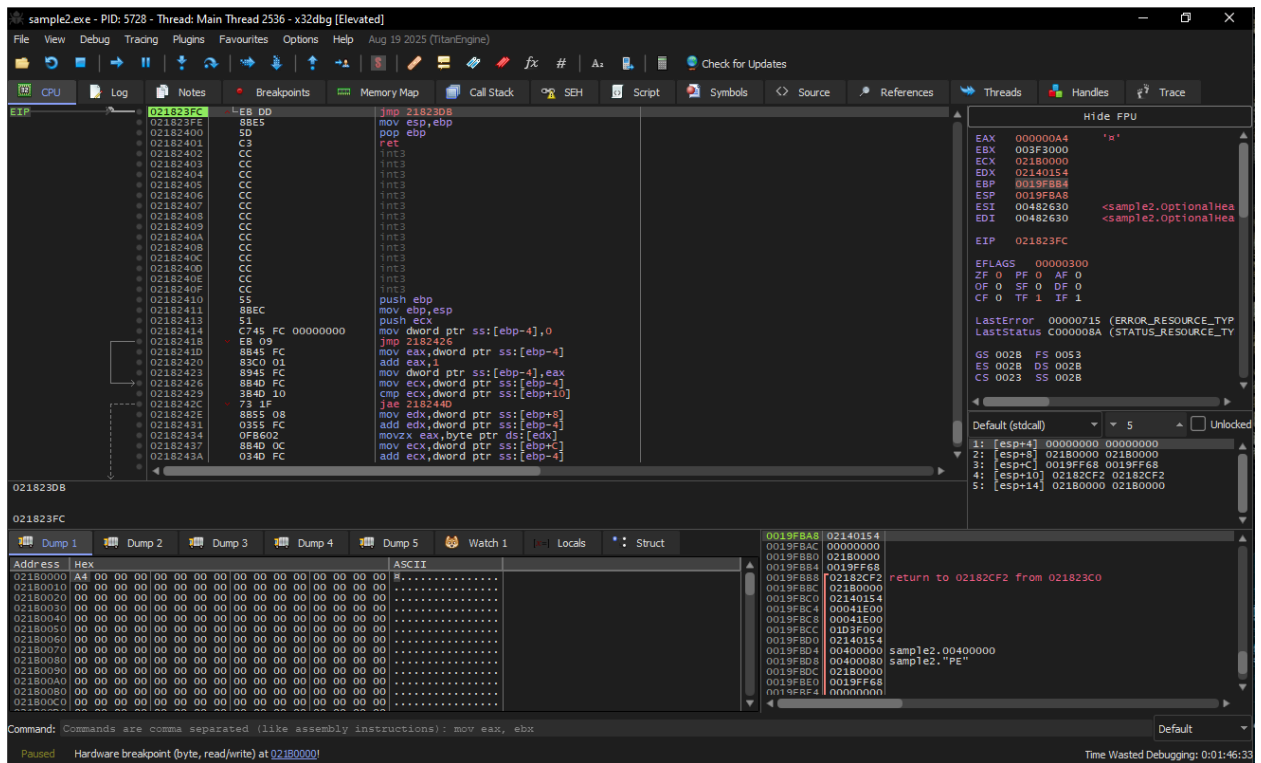
Follow eax in memory dump.



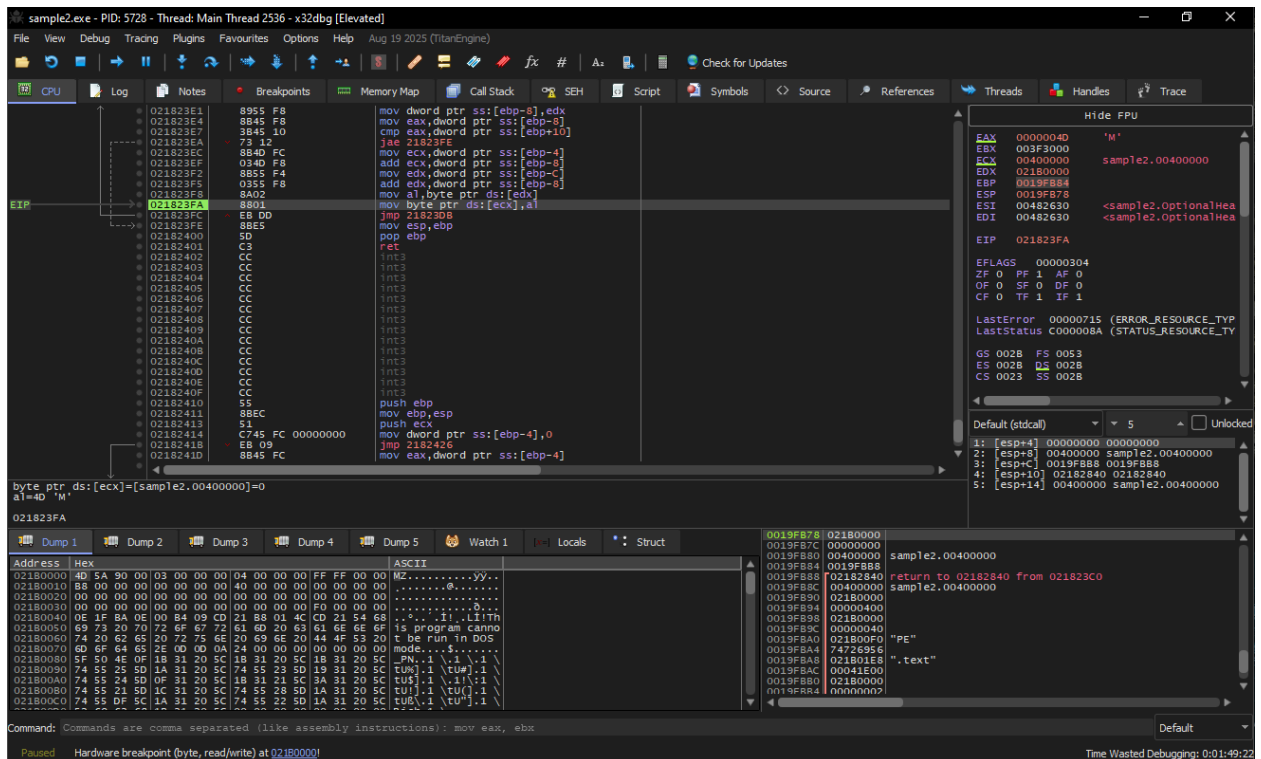
Setting hardware breakpoints.



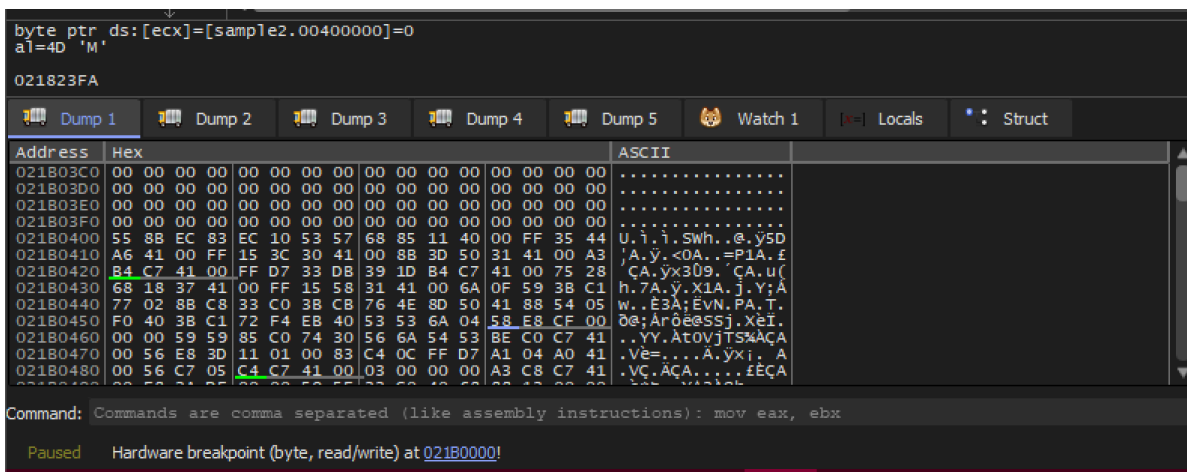
Then f9 we can see the first byte A4



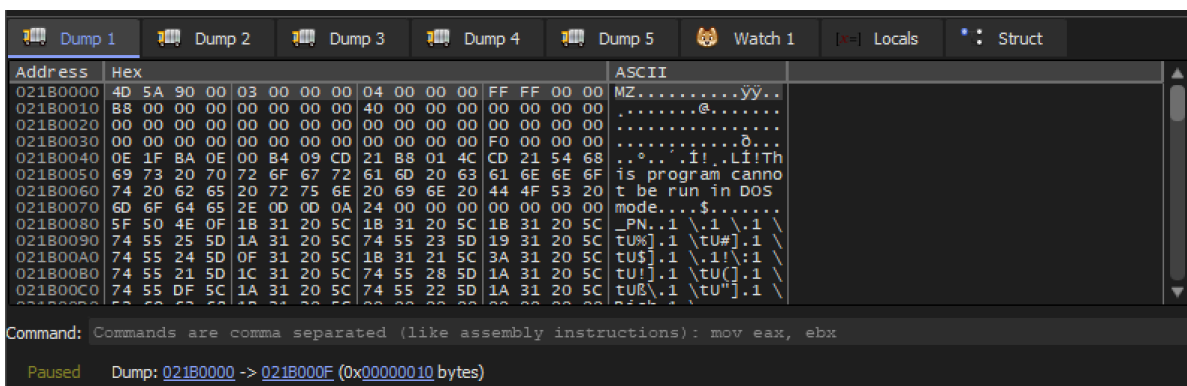
hit f9 couple of times. until you see the MZ header decoded.



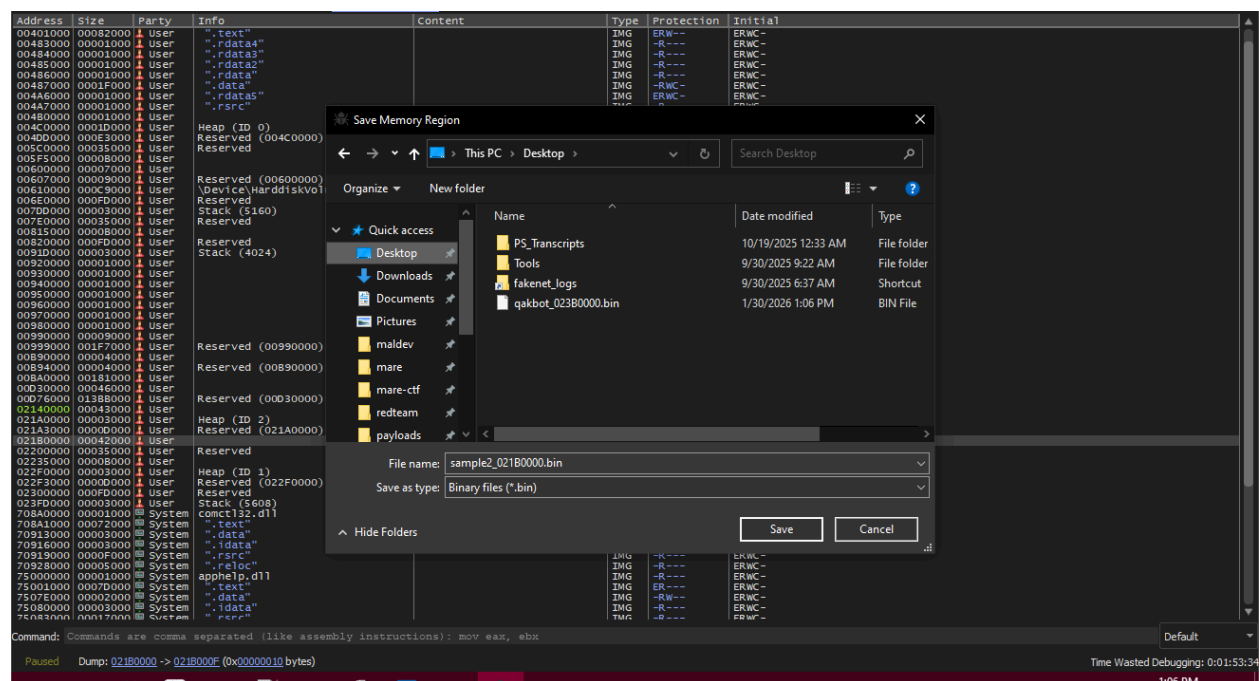
We can see the .text section starts from 400 hex



We will dump this.



Dump memory to file.

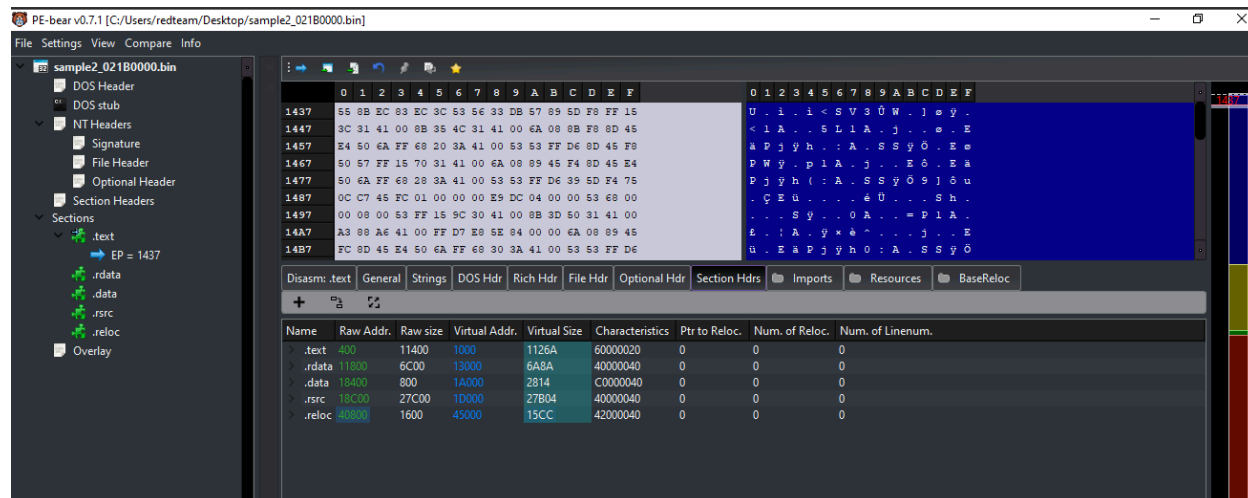


Open the the dumped bin file into pe bear.

last section .reloc

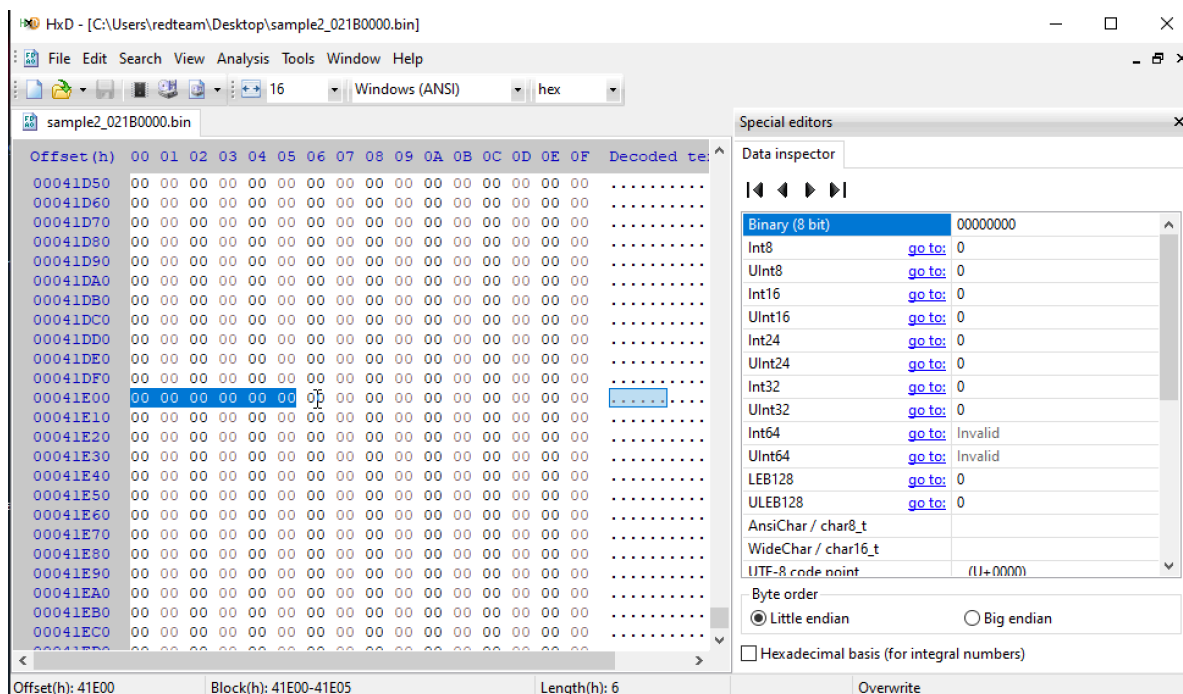
Raw Addr. + Raw size

0x40800 + 0x1600 = 0x41E00



now open the file in hxd and go to offset 0x41E00 to verify.

We can see there are bytes after 41E00 because this unpacked file didn't stopped at the end of the heap.



Select and delete everything after 41E00 and make surer to save it.

